ECM3446/ECMM461:
High Performance Computing

UNIVERSITY OF EXETER

Loop carried dependencies

Unit 2.7

Introduction

- The example **`calc_sum.c`** demonstrated a data race when multiple threads updated a shared variable

- The bug was fixed by using a reduction variable to accumulate the sum instead of a shared variable

- In this unit we will look at other types of race condition which can cause parallel loops to produce incorrect results

Data dependencies

- If
  - one statement reads from or writes to a memory location **and**
  - another statement reads from or writes to a memory location **and**
  - one or both of these accesses is a write
- then the result depends on the order in which the statements are executed and a data dependency exists

Loop carried dependencies

- When there is dependence between statements executed by different iterations of a loop it is called a loop carried dependency
- In this unit we will see examples of three types of loop-carried dependency:
  - Flow dependency (read after write)
  - Anti-dependency (write after read)
  - Output dependency (write after write)

Flow dependency

- Also known as read after write dependency or true dependency
- Iteration `i` requires the result from iteration `i-1`
- The serial loop always updates `a[i-1]` before updating `a[i]` but this will not always happen in parallel
- It may not be possible to remove a flow dependency

```
for (int i=1; i<N; i++){
  a[i] = a[i-1] + a[i];
}
```

Calculating `a[i]` needs `a[i-1]` after it has been updated

Anti-dependency

- Also known as write after read dependency
- Iteration `i` requires that element `i+1` has **not** been updated
- The serial loop always updates `a[i]` before updating `a[i+1]` but iteration `i+1` may take place before iteration `i` in parallel

```
for (int i=1; i<N; i++){
  a[i] = a[i] + a[i+1];
}
```

Calculating `a[i]` needs `a[i+1]` before it is updated

Anti-dependence

- In this case we can write to a different array to ensure iterations are independent

- We can copy `b` to `a` after the loop has completed if required

```
for (int i=1; i<N; i++){
  b[i] = a[i] + a[i+1];
 }
```

Output dependency

- When two statements write to the same memory location there is an output dependency

- In this example each iteration of the first loop writes to x

- The print statement reports the value of x from the sequentially last loop iteration

- In parallel we do not know which loop iteration will write to x last

Calculate $y = \ln(x)$ for x=10,20,…100

```c
#include <stdio.h>
#include <math.h>

int main(){

  const int N=10;
  float x, dx;
  float y[N];

  dx=10.0;
  for (int i=0; i<N; i++){
    x    = (float) (i+1) * dx;
    y[i] = log(x);
  }

  printf("Last x value= %g\n", x);

  for (int i=0; i<N;i++){
    printf("y[%d]=%g\n", i, y[i]);
  }

  return 0;
}
```

Lastprivate variables

- Can we scope **`x`** such that we get the correct behaviour?

- We can scope **`x`** as a **`lastprivate`** variable:

  - Each OpenMP thread has a private copy of a **`lastprivate`** variable (like **`private`**)

  - If a variable is declared **`lastprivate`** then the value from the sequentially last loop iteration is retained when the parallel region ends

```c
#include <stdio.h>
#include <math.h>

int main(){

  const int N=10;
  float x, dx;
  float y[N];

  dx=10.0;
#pragma omp parallel for default(shared) lastprivate(x)
  for (int i=0; i<N; i++){
    x    = (float) (i+1) * dx;
    y[i] = log(x);
  }

  printf("Last x value= %g\n", x);

  for (int i=0; i<N;i++){
    printf("y[%d]=%g\n", i, y[i]);
  }

  return 0;
}
```

Scope x as `lastprivate`

Unit summary

- Data dependencies between loop iterations result in incorrect behaviour if the loop is parallelised

- We looked at three cases:
  - Flow dependence (read-after-write): may not be able to correct without re-thinking the algorithm
  - Anti-dependence (write-after-read): corrected by writing to another array
  - Output dependence (write-after-write): corrected by using a **`lastprivate`** variable